

Intégration déploiement

Docker & github

Finalisons notre architecture

Nous avons maintenant un serveur mysql, un adminer qui expose un manager mysql, et un serveur nodejs qui se connecte à mysql et fourni une API

Il nous manque maintenant une connexion à l'api sur notre projet React et faire tourner la webapp dans un dernier composant, qui communiquera avec notre api

Appel à l'api

A l'aide d'axios (à installer) on va pouvoir faire appel à notre api, en utilisant le port passé en paramètre, qui sera le même que celui du serveur

On affiche dans le dom le nombre d'utilisateur récupéré

```
import './App.css';
import axios from 'axios';
import { useState, useEffect } from 'react';

function App() {
  const port = process.env.REACT_APP_SERVER_PORT;
  let [usersCount, setUsersCount] = useState(0);

  useEffect(() => {
    async function countUsers() {
      try {
        const api = axios.create({
          baseURL: `http://localhost:${port}`
        });
        const response = await api.get(`/users`);
        setUsersCount(response.data.utilisateurs.length);
      } catch (error) {
        console.error(error);
      }
    }
    countUsers();
  }, []);

  return (
    <div className="App">
      <header className="App-header">
        <h1>Users manager</h1>
        <p>{usersCount} user(s) already registered</p>
      </header>
    </div>
  );
}

export default App;
```

Image React

La commande ENV va nous permettre d'utiliser npm sans gérer les droits d'accès des fichiers en explicitant le .bin dans le PATH global

On copie les fichiers package.json et lock, puis on installe les dépendances ainsi que react-scripts pour pouvoir exécuter le npm start plus tard



DockerfileReact

```
1  # pull official base image
2  FROM node:20
3
4  # set working directory
5  WORKDIR /app
6
7  # add `/app/node_modules/.bin` to $PATH
8  ENV PATH /app/node_modules/.bin:$PATH
9
10 # install app dependencies
11 COPY package.json ./
12 COPY package-lock.json ./
13 RUN npm install --silent
14 RUN npm install react-scripts@5.0.1 -g --silent
15
16 EXPOSE 3000
```

Service React

Dans le docker compose on va définir les volumes du dossier courant vers le dossier app sur le container, sauf les node_modules, et utiliser le champ command pour lancer l'app web une fois l'environnement en place

```
react:
  image: react
  build: .
  ports:
    - 3000:3000
  environment:
    - REACT_APP_SERVER_PORT=8000
  volumes:
    - ./app
    - /app/node_modules
  depends_on:
    - server
  command: npm start
```

Lancez votre environnement total

Votre application doit vous afficher le bon nombre d'utilisateur

C'est l'occasion donc d'ajouter un nouveau script de migration qui va insérer au moins un utilisateur à votre table

Builder les images directement depuis le docker-compose

Il est possible de préciser le chemin vers le dockerfile nécessaire aux images de chaque service.

Ainsi il n'est plus nécessaire de supprimer les images et les builder avant de redéployer notre docker compose

Définir le dockerfile

Le champ build va devenir :

```
build:
```

```
  context: .
```

```
  dockerfile: relativePathToDockerFile
```

On va pouvoir faire un `docker-compose -f docker-compose.yml build` pour rebuild toutes les images définies dans le fichier yml

Si les images n'existent pas encore, la commande `up` suffit

A vous de jouer

Modifier votre docker-compose pour préciser le chemin vers les images interne correspondant à un dockerfile

Donc votre service server et votre service react

Optimisez les volumes pour vos services, afin de ne copier que les fichiers nécessaire à la gestion des dépendances, et monter les fichiers et dossiers nécessaire à votre serveur et votre front

Optimisons les volumes

Notre image copie de nombreux fichiers en gérant des droits d'accès

Simplifions nous la vie ! Déplaçons le fichier server.py et dockerfile dans un dossier server

Les volumes vont copier tous le dossier server sauf node_modules

On applique la commande directement dans le docker compose
A vous d'adapter pour le serveur python

```
server:
  image: nodejs
  volumes:
    - ./server:/server
    - /server/node_modules
  build:
    context: .
    dockerfile: ./server/DockerfileNodejs
  ports:
    - 8000:8000
  environment:
    - MYSQL_HOST=db
    - MYSQL_DATABASE
    - MYSQL_USER
    - MYSQL_ROOT_PASSWORD
    - PORT=8000
  depends_on:
    - db
  command: node server.js
```

Simplifions l'image node (DockerfileNodejs)

```
#Create Nodejs Image for JSP Tutorial Application
FROM node:20
WORKDIR /server
ENV PATH /server/node_modules/.bin:$PATH
RUN npm install express cors mysql2 dotenv --silent
EXPOSE 8000
```

On test nos modifications

```
docker-compose -f docker-compose.yml build
```

```
docker-compose -f docker-compose.yml up --detach
```

.dockerignore

Le fichier Docker .dockerignore est utilisé pour éviter que certains fichiers et répertoires ne se retrouvent dans votre Docker image.

Le fichier Docker .dockerignore est utilisé lorsque vous construisez un Docker image et consiste en des règles correspondant à des fichiers et/ou des répertoires.

- Réduction de la taille du fichier Docker image
- Plus rapide Docker image construit
- Éviter la divulgation des secrets



.dockerignore

```
1  node_modules
2  build
3  .dockerignore
4  Dockerfile
5  /server/DockerfileNodejs
6  DockerfileReact
7  /server/node_modules
```

Github actions

On va tester le déploiement de notre application dans un docker compose au sein de notre pipeline github

- name: Build the stack

- run: docker compose up -d

L'objectif ? Pouvoir exécuter des test end2end sur notre architecture

Et à terme, déployer cette architecture vers un service d'hébergement docker payant comme digitalOcean, aws, hostinger, kamatera, ou vers un serveur perso

Exemple de déploiement avec aws ec2

```
name: Deploy
on:
  push:
    branches: [ main ]

jobs:
  EC2-Deploy:
    runs-on: ubuntu-latest
    environment:
      name: ${{ github.ref_name }}
      url: ${{ steps.deploy.outputs.vm_url }}
    steps:
      - id: deploy
        name: Deploy
        uses: bitovi/github-actions-deploy-docker-to-ec2@v0.5.0
        with:
          aws_access_key_id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws_secret_access_key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws_default_region: us-east-1
          app_port: 3000
```

source : <https://www.bitovi.com/blog/deploy-any-docker-project-to-aws-with-github-actions>